

AI Request Triage Assistant

Technical design implementation and evaluation report

A local, review-first agent workflow for professional-services request classification, priority reasoning, ownership routing, and safe customer-response drafting.

Submission	Implementation
Candidate	Akshita Rastogi
Challenge	Node Solutions AI Technical Challenge
Runtime	Docker, FastAPI, Streamlit, PostgreSQL, Redis
AI	Local Ollama with two schema-constrained roles
Orchestration	LangGraph typed state workflow

The system supports the six supplied mock requests and new mock requests. Every output is a draft for human review; no endpoint sends messages or changes a customer system.

Executive summary

The implementation satisfies the assessment while adding targeted controls for the failure modes that matter most in real AI workflows.

The assistant accepts a written request and returns a short summary, exactly one category, one priority with a reason, one allowed owner, and a professional first-response draft. The six supplied messages are data fixtures rather than hardcoded response branches; the same endpoint accepts any validated mock request between 5 and 5,000 characters.

The main engineering decision is to separate model judgment from business safety. Ollama handles summarisation and nuanced classification through a strict JSON schema. LangGraph then applies deterministic policy before drafting. Privacy exposure, outages, billing, commercial intent, and product enhancements therefore have stable routing invariants.

Results at a glance

Dimension	Implemented approach	Why it matters
Local AI	Ollama model on the host	No hosted inference key or cloud data transfer
Reliability	Schema validation, repair, and fallback	Malformed output cannot break the contract
Safety	Redaction, injection branch, policy rules	High-cost cases do not rely only on prompting
Review	Immutable decision plus separate review	AI history is never overwritten
Operations	Idempotency, rate limits, health, metrics	Retries and failures are visible
Evaluation	Golden cases plus adversarial tests	Routing regressions are reproducible

Assessment requirements and coverage

Each requested capability maps to an observable implementation and a concrete demonstration path.

Assessment requirement	Implementation	Demonstration evidence
Written request	Streamlit form and POST /v1/triage	Use any supplied or custom mock text
Short summary	AnalysisDecision.summary	Visible above category metrics
Five categories	Strict Category enum	Invalid values fail schema validation
Four priorities	Strict Priority enum plus rationale	Priority and reason appear together
Allowed owners	Strict Owner enum and policy rules	Owner shown in UI and database
First response	Separate response role or guarded template	Always labelled for review
Six requests	data/mock_requests.json	All six appear in the selector
New requests	Generic typed endpoint	Six extra edge cases plus custom input
Request 05	Privacy incident invariant	Urgent Engineering route and risk flags
Decisions and limitations	README and report	Trade-offs are stated explicitly

Expected routing for the supplied requests

ID	Primary intent	Category	Priority	Owner
01	Automation discovery call	Sales	Medium	Sales Team
02	Active portal outage	Technical	Urgent	Engineering
03	Duplicate invoice charge	Billing	High	Finance
04	Future product enhancement	Technical	Low	Engineering
05	Wrong-workspace data exposure	Technical	Urgent	Engineering
06	Custom AI system enquiry	Sales	Medium	Sales Team

03

Architecture

The design uses a small number of components with explicit responsibilities and stable boundaries.

01 Streamlit Input and review	02 FastAPI Validate and control	03 LangGraph Four typed stages	04 Ollama Local AI roles	05 PostgreSQL Decision and review
---	---	--	--	---

The browser talks only to FastAPI. The API authenticates, validates, rate-limits, and checks idempotency before invoking the graph. LangGraph passes typed state through sanitation, analysis, validation, and drafting. PostgreSQL stores the final decision and human review; Redis stores only short-lived rate counters. Structured logs and Prometheus metrics observe the HTTP and decision paths without using request text as labels.

Component choices

Component	Responsibility	Decision rationale
Streamlit	Operator input, results, trace, review queue	Fast local UI with no production UI requirement
FastAPI	Typed HTTP boundary and status semantics	Async, OpenAPI-native, integrates with Pydantic
LangGraph	Explicit state and ordered node execution	Control flow and failure paths remain visible
Ollama	Local analysis and response generation	Private local inference and structured output
PostgreSQL	Durable decisions and human reviews	Uniqueness, foreign keys, ordering, and audit integrity
Redis	Shared rate-limit counters	Expiry and future multi-replica consistency

Why there is no vector database

This challenge is classification and drafting, not knowledge retrieval. There is no policy corpus to embed or cite. Adding Qdrant would create operational surface without improving the requested result. If policy documents are introduced, a retrieval node can be inserted before analysis and carry excerpts and citations in graph state.

LangGraph workflow and state

The graph is an auditable state machine, not an open-ended autonomous agent.

Node	Input	Output	Failure behavior
Sanitize	Original request	Redacted text, injection flag	Reject invalid API input before graph
Analyse	Redacted text	Typed AnalysisDecision	One repair, then deterministic fallback
Validate	Decision and request signals	Policy-corrected decision	Non-negotiable invariants win
Draft	PII-free summary and decision	Customer-facing draft	Repair, template, or guarded branch

State fields

Field	Producer	Purpose
original_text	Service entry	Faithful controlled audit input
redacted_text	Sanitize	Only request form eligible for analysis inference
injection_detected	Sanitize	Conditional safety decision
decision	Analyse and Validate	Schema-safe classification and routing
decision_source	Analyse or injection policy	Distinguishes model, fallback, and guardrail
warnings	All relevant stages	Add why the path changed or degraded
trace	Every graph node	Attempts, duration, tokens, corrections, and branches
draft	Response agent or template	Review-only first response

Warnings and trace use additive reducers. Each node appends its evidence rather than overwriting prior stages. The final UI trace therefore explains the complete route.

Conditional safety branches

- Prompt injection: normalise to Other, Low, Client Success and use a guarded clarification template.
- Privacy or access-control incident: preserve Urgent Engineering routing and use a deterministic incident acknowledgement that never claims remediation occurred.
- Normal request: use the response agent with a PII-free validated summary.

Why LangGraph instead of other frameworks

The selected framework matches a known, safety-ordered workflow and leaves room for controlled expansion.

Option	Strength	Reason for this decision
LangGraph	Typed state, reducers, conditional edges, interrupts	Selected: control flow and evidence are explicit
LangChain	Model, prompt, retrieval, and tool composition	Useful ecosystem, but a linear chain hides state transitions
ReAct agent	Dynamic tool and step selection	Rejected: unnecessary autonomy and variable latency
CrewAI or AutoGen	Conversational multi-agent teams	Rejected: two ordered roles do not need agent messaging
Plain Python	Minimal dependencies and direct control	Viable now, but future branches and interrupts are less visible

The workflow is sequential because each stage depends on the prior result: sanitation must precede inference, validation must follow analysis, and drafting must use the validated decision. Parallel agents would add coordination cost without shortening this critical path.

Why two AI roles

- The analysis role uses temperature 0 and a Pydantic JSON schema for stable routing fields.
- The response role uses temperature 0.1 and a DraftOutput schema for constrained language.
- Separating them prevents customer-facing tone generation from changing category, priority, or owner.

End to end code execution

This section follows one request from the browser to durable storage and human review.

1. Docker starts uvicorn triage.main:app. FastAPI lifespan opens PostgreSQL and Redis clients, creates the Ollama client, compiles the graph once, and constructs TriageService.
2. Streamlit api() sends POST /v1/triage with request text, source, API key, and a unique idempotency key.
3. request_context() binds a correlation ID, measures latency, records status metrics, and writes a structured completion log.
4. TriageRequest validates length, source, meaningful content, and normalised whitespace. require_api_key() authenticates without logging the key.
5. RateLimiter.allow() increments a Redis minute bucket. TriageService checks PostgreSQL for a prior successful result with the same idempotency key.
6. The sanitize node redacts contact and payment identifiers, detects injection language, and appends warnings and trace evidence.
7. The analysis node calls OllamaClient.analyse(). Ollama receives the AnalysisDecision JSON schema; Pydantic validates the returned values. Invalid output gets one repair attempt.
8. The validation node applies incident, outage, billing, feature, and commercial ownership invariants. It records any correction visibly.
9. The drafting stage sees only the validated summary and decision. High-risk and injection branches use deterministic templates; normal requests use structured local generation.
10. Database.insert() writes one complete result. The API returns HTTP 201, or HTTP 200 with Idempotent-Replayed for a replay.
11. show_result() renders the decision and trace. A human review is written to triage_reviews without changing the original inference.

Code structure and method guide

Each module has one primary reason to change.

File	Main symbols	Role
main.py	lifespan, middleware, routes	HTTP boundary, lifecycle, metrics, status codes
models.py	enums, request, decision, result	One source of truth for typed contracts
graph.py	TriageState, build_graph	Nodes, edges, conditional safety paths
ollama.py	health, analyse, draft	Local inference, schemas, timeout, repair
guardrails.py	safe_fallback, enforce_business_rules	Model-independent routing policy
security.py	redaction, injection, authentication	Untrusted-input boundary
service.py	TriageService.triage	Idempotency, graph invocation, persistence
database.py	connect, insert, get, list, review	PostgreSQL schema and repository
rate_limit.py	allow, healthy, close	Redis fixed-window operational control
metrics.py	REQUESTS, LATENCY, DECISIONS	Low-cardinality Prometheus telemetry
ui/app.py	api, show_result	Input, history, queue, evaluation, explanation
evaluate.py	main	Golden set through the real HTTP API

Adding new behavior safely

A new request needs no ingestion: paste mock text or call the endpoint. To add a demo fixture, append an object to `edge_case_requests.json`. To add a category, update the enum, prompt, policy rules, UI expectations, golden cases, and tests together. To add a tool, define typed input and output, a conditional graph edge, timeout and retry limits, and persisted trace evidence.

Reliability privacy and guardrails

Controls are layered because no single prompt or confidence value is a sufficient safety boundary.

Risk	Control	Visible evidence
Malformed model JSON	Pydantic schema plus one repair	Attempts and fallback warning
Model unavailable	Deterministic decision and draft	decision_source and trace status
Prompt injection	Detection and conditional isolation	prompt_injection flag and guarded template
PII leakage	Pre-inference redaction and summary-only drafting	Redaction types in warnings and trace
Under-prioritised incident	Security and outage invariants	Policy correction warning
False remediation claim	Security response template and draft validator	No completion claim reaches UI
Duplicate client retry	PostgreSQL idempotency uniqueness	HTTP 200 replay header
Abuse or burst	Redis shared fixed-window limit	HTTP 429 and expiring counter
Hidden dependency failure	Readiness probes	HTTP 503 with per-dependency state

Draft-specific validation

- Reject reasoning markers such as 'the user is asking' or 'I need to'.
- Reject redaction placeholders and instructions to contact a placeholder.
- Reject unverified claims such as 'we have confirmed', 'we initiated', or 'we removed'.
- Reject Urgent or immediate language when the validated priority is not Urgent.
- Require a complete 20-800 character structured message and terminal punctuation.

Additional request coverage

Six extra cases demonstrate generalisation and adversarial handling beyond the supplied fixtures.

Case	What it tests	Expected behavior
E01 Ambiguous	Insufficient specialist evidence	Other and human clarification
E02 Injection	Embedded prompt instructions	Isolate, flag, and guarded response
E03 Identifiers	Email and phone plus billing	Redact, retain Billing High Finance
E04 Conflict	Customer says not urgent but portal is down	Impact wins: Technical Urgent Engineering
E05 Mixed intent	Billing and sales in one message	Choose one primary operational category
E06 Novel support	Unseen account-administration question	Support and Client Success

These examples are not proof of universal coverage. Novel languages, subtle legal requests, and unseen security paraphrases require a larger reviewed dataset. Human review remains the final safety boundary.

Persistence observability and operations

The local deployment can be inspected through both terminal commands and desktop database tools.

PostgreSQL records

Table	Stored information	Integrity
triage_requests	Input, redacted form, decision, draft, warnings, trace	UUID primary key and unique idempotency key
triage_reviews	Outcome, notes, review timestamp	One review per request with foreign key

DBeaver connection: host localhost, port 5433, database triage, username triage, password triage. RedisInsight connection: host localhost, port 6380, no username or password. Redis rate keys expire after 60 seconds, so refresh immediately after an API request.

Operational commands

```
curl -s http://localhost:8100/health/ready | python -m json.tool
docker compose logs --tail=100 api
curl -s http://localhost:8100/metrics | grep '^triage_'
docker compose exec redis redis-cli --scan --pattern 'rate:*
```

HTTP semantics

Status	Meaning
200	Read operation or idempotent replay
201	New triage result or review
401	Missing or invalid API key
404	Unknown request ID
409	Duplicate review or idempotency race
422	Invalid body, enum, UUID, or limit
429	Rate limit exceeded
503	Required dependency is unavailable
500	Unexpected internal failure with safe public detail

Evaluation strategy

Evaluation checks deterministic routing today and defines the measurements needed before a real rollout.

data/evaluation.json contains the six supplied requests with expected category, priority, and owner. scripts/evaluate.py calls the real HTTP boundary, compares all three fields, and writes detailed expected-versus-actual evidence to output/evaluation-results.json.

Metric	Purpose	Release interpretation
Category macro F1	Balance performance across categories	Avoid majority-class accuracy
Owner exact match	Verify operational routing	Wrong team is a workflow failure
Critical false-negative rate	Measure missed outages/privacy cases	Release-blocking safety metric
Schema-valid rate	Track structured-output reliability	Should remain near 100 percent
Human edit distance	Approximate draft usefulness	Lower is better after reviewer agreement
p50 p95 p99 latency	Expose slow-tail behavior	Regression gate by environment
Fallback rate	Detect model or prompt degradation	Investigate sustained increases

Commands

```
docker compose exec api pytest -q
docker compose exec api ruff check --line-length 100 src tests scripts ui
docker compose exec api python scripts/evaluate.py
python -m json.tool output/evaluation-results.json
```

The current model confidence is diagnostic metadata, not a calibrated probability and not permission to auto-send. A larger evaluation set would be versioned, multilingual, balanced, paraphrase-heavy, and independently labelled by multiple reviewers.

Additional engineering features

The extensions focus on trustworthy operation, not decorative feature count.

Feature	Engineering value
Typed agent state	Separate roles and additive trace reducers make every transition auditable.
Structured local output	Both Ollama roles return Pydantic-validated JSON with bounded repair.
Deterministic policy	High-cost routing is enforced outside probabilistic model behavior.
Data minimisation	The drafting role sees only a validated PII-free summary and decision.
Conditional guardrails	Injection and privacy paths bypass normal generation when autonomy adds risk.
Graceful degradation	Local model failure returns safe, visible fallback behavior.
Immutable review	Human feedback is captured without rewriting the historical model output.
Safe retry semantics	Idempotency and database uniqueness prevent duplicate successful work.
Operational visibility	Correlation logs, dependency checks, metrics, and graph traces aid diagnosis.
Adversarial evaluation	Tests intentionally challenge routing, PII, injection, and reasoning leakage.

Limitations and next improvements

The project is production-aware but does not claim that a local assessment application is production-ready.

Known limitations

- The deterministic vocabulary is English and finite; new paraphrases need evaluated policy updates.
- Six golden examples cannot estimate real generalisation, class imbalance, or multilingual quality.
- The demo uses one API-key role and has no tenant row-level security or secrets manager.
- Model confidence is not calibrated against held-out reviewer labels.
- The fixed-window limiter can permit a boundary burst, although it is shared across replicas.
- No endpoint sends email, edits workspace access, or performs remediation in customer systems.

Next improvements

1. Expand the versioned dataset with multilingual, ambiguous, conflicting, and reviewer-disagreement cases.
2. Calibrate critical-risk scores and add release gates for false negatives, schema validity, and latency percentiles.
3. Add role-based access, tenant isolation, managed secrets, migrations, and encrypted audit retention policies.
4. Introduce OpenTelemetry spans and dashboards for per-node latency, model fallback rate, and reviewer corrections.
5. Use reviewed corrections as offline evaluation or supervised-learning data only after governance approval.

Capturing approval or rejection is not reinforcement learning or RLHF by itself. Feedback becomes useful only after curation, quality review, and a separate offline improvement process.

Demonstration guide

A short demonstration should show correctness, explainability, a high-risk branch, persistence, and evaluation rather than every screen control.

Time	Screen	What to demonstrate
0:00-0:40	README and architecture	Problem, local stack, four graph stages, and review-first boundary
0:40-1:30	Request 01	Sales correction, structured draft, node attempts, tokens, and latency
1:30-2:15	Request 02	Impact-driven Urgent Engineering outage routing
2:15-3:10	Request 05	Privacy flags and deterministic incident response without false remediation
3:10-3:55	Edge E03	Email and phone redaction plus Billing High Finance decision
3:55-4:30	Edge E02	Conditional injection isolation and guarded clarification template
4:30-5:15	Review queue and DBever	Immutable decision and separate human review rows
5:15-5:50	RedisInsight and metrics	Expiring hashed rate key and low-cardinality telemetry
5:50-6:30	Evaluation	Golden expected-versus-actual checks and honest limitations

Concise narration

I built a local, review-first request triage assistant. FastAPI validates the request and applies API-key, rate-limit, and idempotency controls. LangGraph then sanitises input, asks Ollama for a schema-constrained decision, enforces deterministic routing policy, and creates a reviewed draft. PostgreSQL preserves both the AI result and a separate human decision. The most important addition is defense in depth: privacy incidents, outages, prompt injection, PII, and false remediation claims are handled outside model optimism. The evaluation harness runs all six supplied cases through the real API and records field-level evidence.

Local runbook

The Docker ports are intentionally separate from the SupplyGuard project so both applications can coexist.

Start

```
cd /Users/akshitarastogi/Documents/Codex/2026-09-09/si/outputs/node-solutions-ai-triage
ollama pull qwen3:4b
cp .env.example .env
docker compose up -d --build
docker compose ps
```

Surface	Address
Streamlit	http://localhost:8601
FastAPI documentation	http://localhost:8100/docs
Prometheus metrics	http://localhost:8100/metrics
PostgreSQL	localhost:5433 / database triage
Redis	localhost:6380
Ollama host	localhost:11 434

Inspect and stop

```
curl -s http://localhost:8100/health/ready | python -m json.tool
docker compose logs -f api
docker compose down
```

`docker compose down` preserves the named PostgreSQL and Redis volumes. Use `down -v` only for an intentional destructive reset.

Conclusion

The solution meets the challenge with a deliberately bounded and explainable agent system.

The assistant is more than a classification prompt: it is a typed workflow with local inference, deterministic safety policy, controlled degradation, durable audit history, and human review. Additional complexity is justified by observable failure modes rather than by an attempt to imitate a large production platform.

The design can evolve without changing its central contract. Retrieval can be added when policy documents exist; specialist tools can be added behind typed conditional edges; and reviewer corrections can expand a governed offline evaluation set. Until then, the current boundaries keep the assessment simple enough to explain and strong enough to demonstrate engineering judgment.